API Access via Software Passkeys

Perry Kundert

2026-08-08

A WebAuthn assertion is a signature over `authenticatorData || SHA-256(clientDataJSON)`. Producing one needs a private key and about sixty lines of code – no browser, no human, no biometric. This memo specifies how a person whose Passkey lives in the macOS Passwords keystore provisions a *second* Passkey for a headless script, where every byte of that exchange flows, and what each side must remember afterwards.

The human's Passkey is a synced platform credential, reachable only through `navigator.credentials.get()`. It signs one kind of object and can never sign on the script's behalf; its role is to *authorize*, once, the creation of the API credential. That credential's private key is generated in the browser page, displayed exactly once, and pasted into the script's `.env`. One piece of private key material, in one place. The server holds only a COSE public key – it never possesses a secret that authenticates.

At runtime the script performs the full WebAuthn ceremony in software, and *the same key* signs the per-request DPoP proofs (RFC 9449). That single-key choice yields the central result: the server stores **nothing** per session beyond the ordinary `sessions` row. The DPoP key thumbprint is *derived* from `credentials.public_key`, not stored, so the per-request check reduces to "this proof was signed by the same key that produced the assertion that opened this session." The only new server state in the entire design is a `jti` replay cache – structurally the same row the challenge table already is.

Contents

1 Status

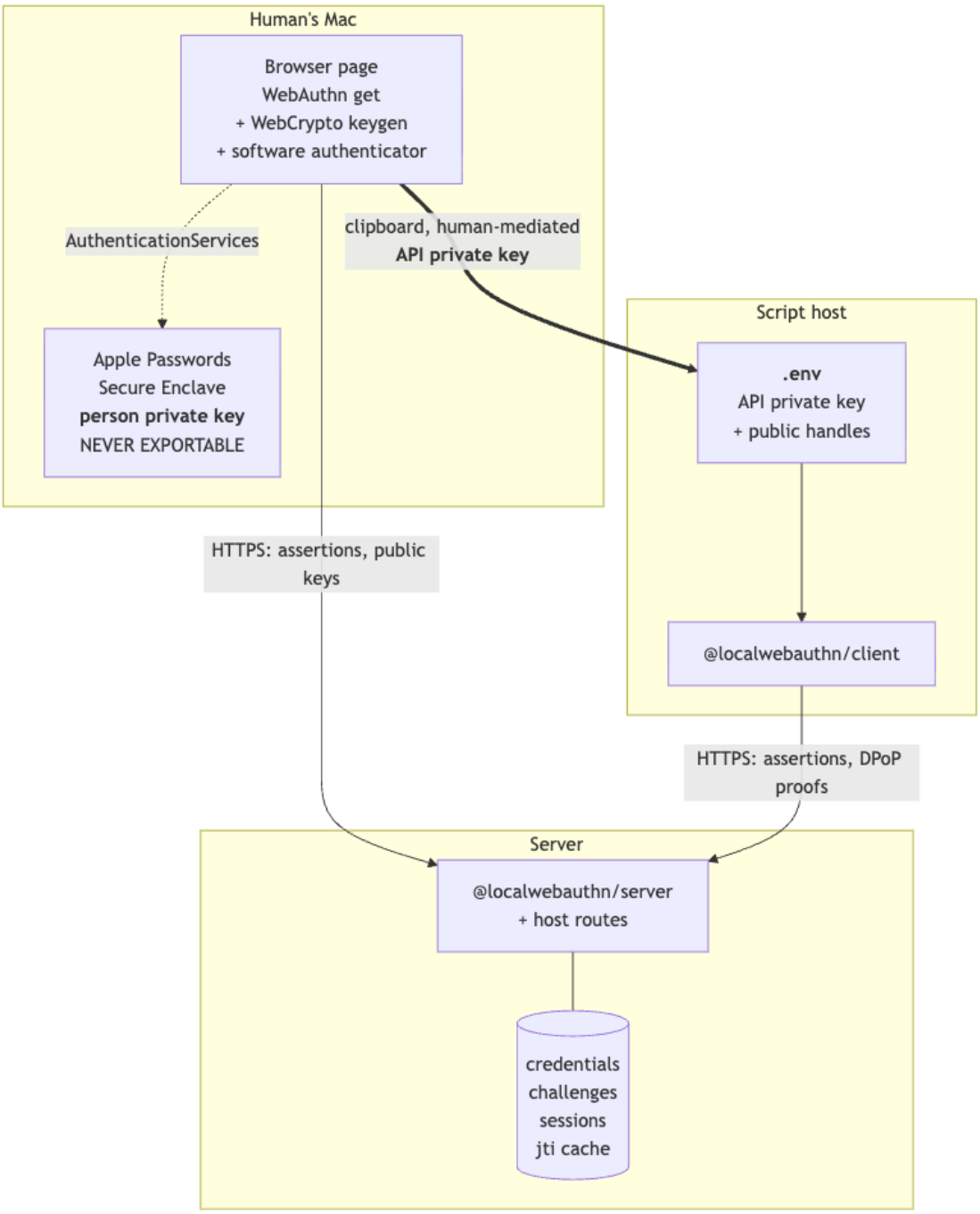
Design document. Describes the intended system, not the shipped one.

As of `@localwebauthn/server@2.2.0`, `Credential.label` and `Credential.lastUsedAt` exist and do everything this design needs of them. The `kind` column does not exist (issue #11), there is no `@localwebauthn/client` package, and there is no DPoP support. Items marked *proposed* are new work; everything else is a call that exists today.

Companion documents in `docs/API-AUTH/`: three independent design treatments, a consensus analysis with the standards survey, and the provisioning discussion this document supersedes.

2 Overview

2.1 Principals



The thick edge is the only path the API private key ever travels: *page memory -> clipboard -> file*. It does not pass through the server, and the server has no column that could hold it.

2.2 The two credentials

Property	Human Passkey	API Passkey
Key custody	Apple Passwords, Secure Enclave	`.env` on the script host
Reachable from	a browser, only	any process that can read `.env`
`kind` (*proposed*)	`person`	`service`
`label`	"Perry's MacBook"	"Perry's Blah maintenance script"
`deviceType`	`multiDevice`	`singleDevice`
`backedUp`	`true`	`false`
`signCount`	`0` forever; iCloud keeps no counter	`0` by policy
`userVerified`	`true`, attested by Touch ID	`true`, self-asserted by a program
`origin` in clientData	written by the *browser*	written by the *script*
Signs	assertions, browser-mediated	assertions + DPoP proofs

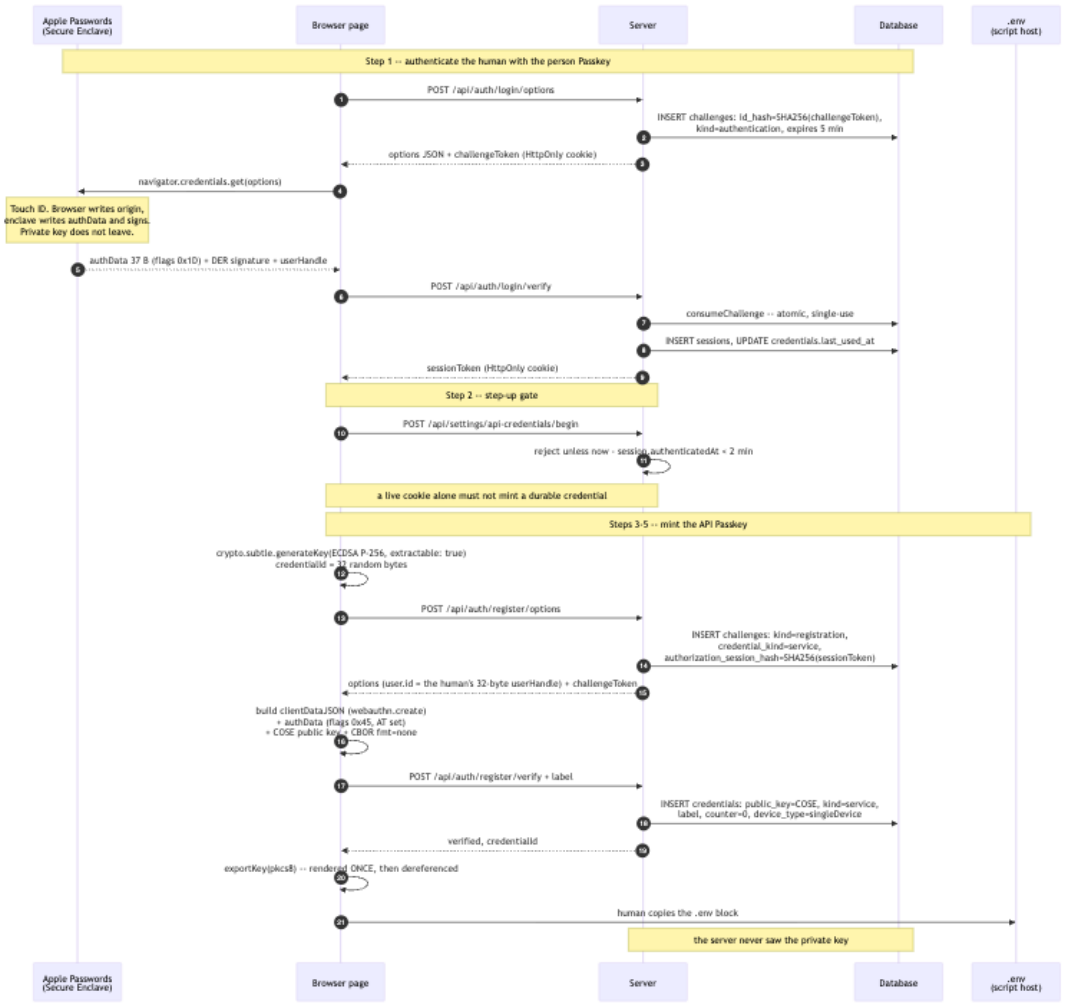
Both rows report `userVerified: true`, and it means something entirely different on each. That is what `kind` exists to record.

3 Part 1: A Human Provisions an API Passkey

3.1 Preconditions

- Server configured `rpId = app.example.com`, `expectedOrigins = ['https://app.example.com']`.
- The human holds a `kind`'person'= credential, private key in Apple Passwords.
- `getUser(userId)` returns them `active: true` with a stable 32-byte `webAuthnUserHandle`.

3.2 End-to-end data flow



3.3 Step 1: authenticate the human

Runs on	browser -> macOS -> Secure Enclave -> server
Code	`LocalWebAuthnBrowser.signIn()`, `auth.authenticationOptions()`, `auth.verifyAuthentication()`

The layer split *is* the phishing resistance, and it is why this key can never serve the script:

`BROWSER writes clientDataJSON:`

```
  {"type":"webauthn.get",
   "challenge":"<echoed verbatim from options>",
   "origin":"https://app.example.com",     <-- the BROWSER asserts this
   "crossOrigin":false}

AUTHENTICATOR writes authenticatorData, 37 bytes:
  [0..32)  SHA-256("app.example.com")
  [32]     0x1D = UP|UV|BE|BS
  [33..37) 0x00000000  signCount

AUTHENTICATOR signs, inside the enclave:
  ES256( authenticatorData || SHA-256(clientDataJSON) )   -- ASN.1 DER
```

Server verification, each step able to fail the call: consume the challenge atomically; look up the credential; require the user `active`; compare `response.userHandle` against `user.webAuthnUserHandle`; verify challenge, origin, `rpIdHash`, the UV bit and the signature; apply the `0 -> 0` counter rule; then commit the counter advance, the `last_used_at` touch and the session insert as one transaction.

3.4 Step 2: the step-up gate

```
const resolved = await auth.resolveSession(sessionToken);
if (!resolved) return unauthorized();

// SECURITY.md: a fresh assertion is required for sensitive credential changes.
// Minting a durable API credential is exactly that.
if (Date.now() - resolved.session.authenticatedAt > 2 * 60_000) {
  return json({ error: 'reauth_required' }, 401);   // browser repeats Step 1
}
```

`authenticatedAt` already exists on `SessionIdentity`, so this costs nothing. Without it, a stolen session cookie mints a credential that outlives the cookie you would revoke: the session becomes a credential factory.

3.5 Step 3: the page generates the API key pair

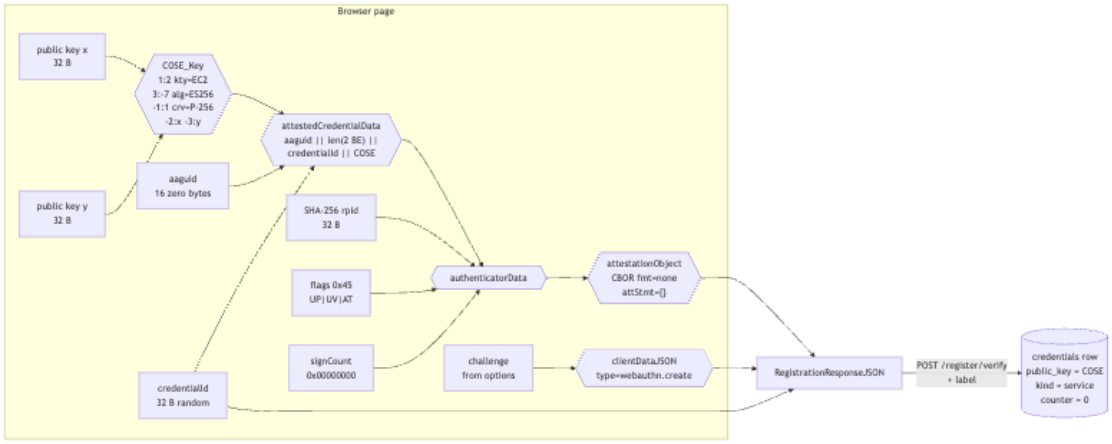
Calling `navigator.credentials.create()` here would mint a key *inside Apple Passwords* that can never be exported. What is wanted is an extractable software key, so the page implements a software authenticator itself:

```
const { publicKey, privateKey } = await crypto.subtle.generateKey(
  { name: 'ECDSA', namedCurve: 'P-256' },
  true,                                    // extractable -- the entire point
  ['sign'],
);
const credentialId = crypto.getRandomValues(new Uint8Array(32));
```

3.6 Step 4: assembling the registration response

The `credentialKind` is supplied by the *host route*, never read from the request body, and is written to the challenge row – so the credential's class is fixed on a server-owned row before the page ever

sees the challenge.



Two things worth noticing. `BE` and `BS` are deliberately clear, so the server records `singleDevice` / `backedUp: false` – honest for a key in a file. And with `fmt: "none"` there is **no attestation signature to compute at all**: the public key is simply asserted, and the first assertion is what proves the private half exists.

3.7 Step 5: one-time display, then transfer

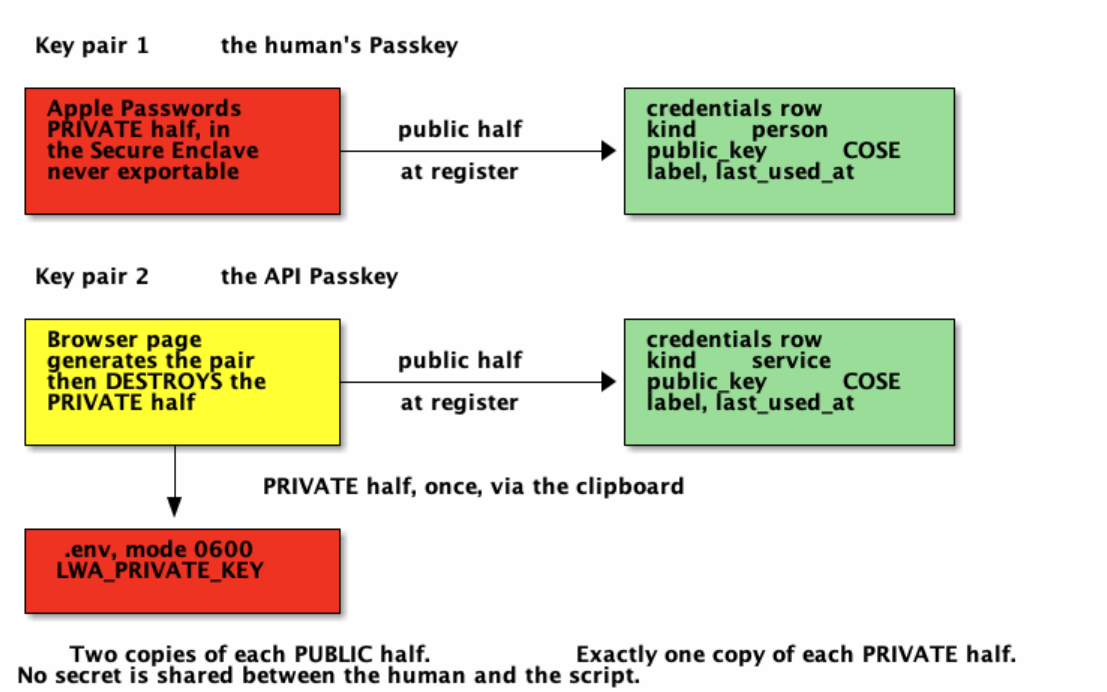
```
# public -- needed to build an assertion, secret to nobody
LWA_BASE_URL=https://app.example.com
LWA_RP_ID=app.example.com
LWA_ORIGIN=https://app.example.com
LWA_CREDENTIAL_ID=k7Rz...            # base64url, 32 bytes
LWA_USER_HANDLE=9fQ2...              # base64url, 32 bytes
LWA_ALG=-7                           # COSE ES256

# the one secret
LWA_PRIVATE_KEY=MIGHAgEAMBMGByqGSM49...   # base64 PKCS#8
```

The familiar "copy it now, you will not see it again" contract – with the improvement that, unlike a conventional API key, the server never had it to begin with. Mode `0600`, gitignored.

If the script host *is* the same Mac, `LWA_PRIVATE_KEY` can be replaced by `LWA_KEY_REF=keychain:...` with the key imported into the login keychain, or into the Secure Enclave – P-256 only, non-exportable – at which point `.env` holds no secret at all.

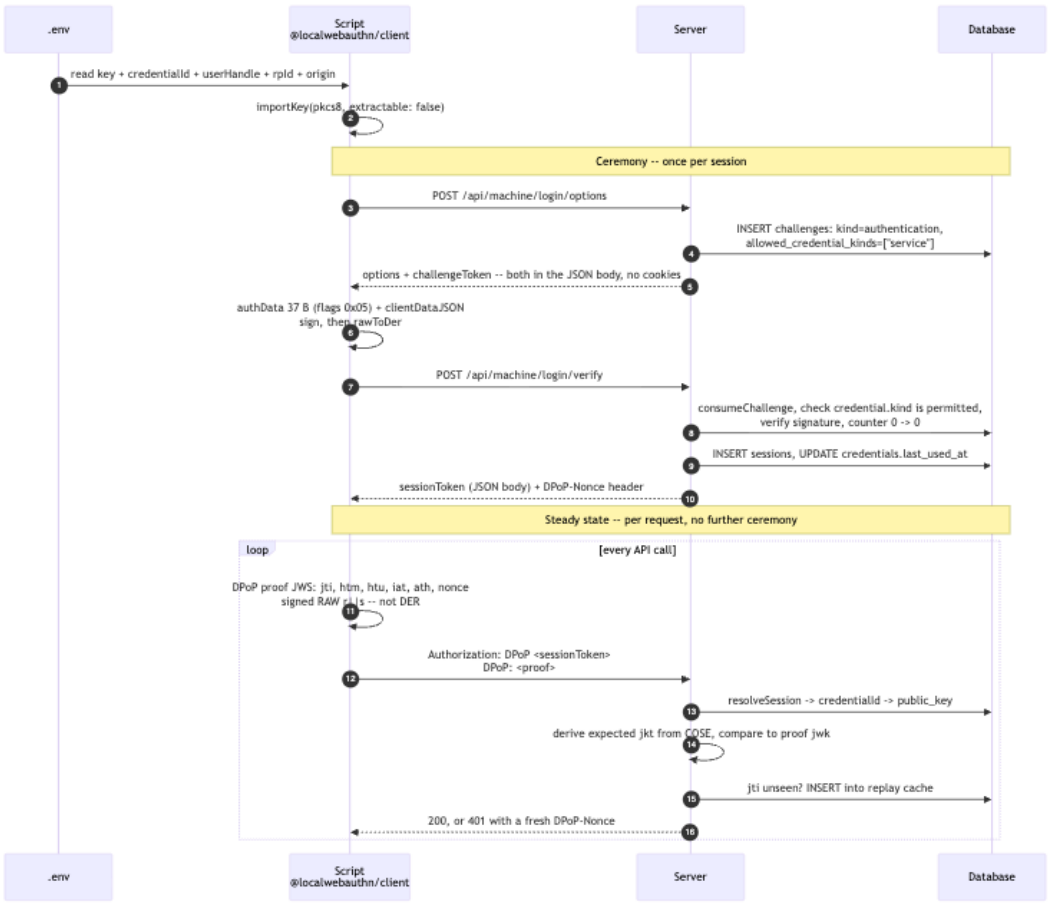
3.8 Where the key material ends up



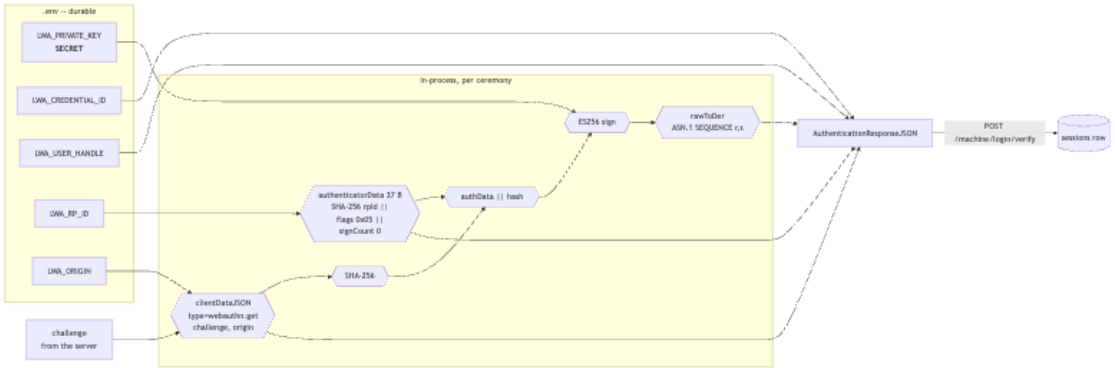
4 Part 2: The Script Opens an API Session

Each session is independent. The same `.env` credential can open any number of them concurrently.

4.1 End-to-end data flow

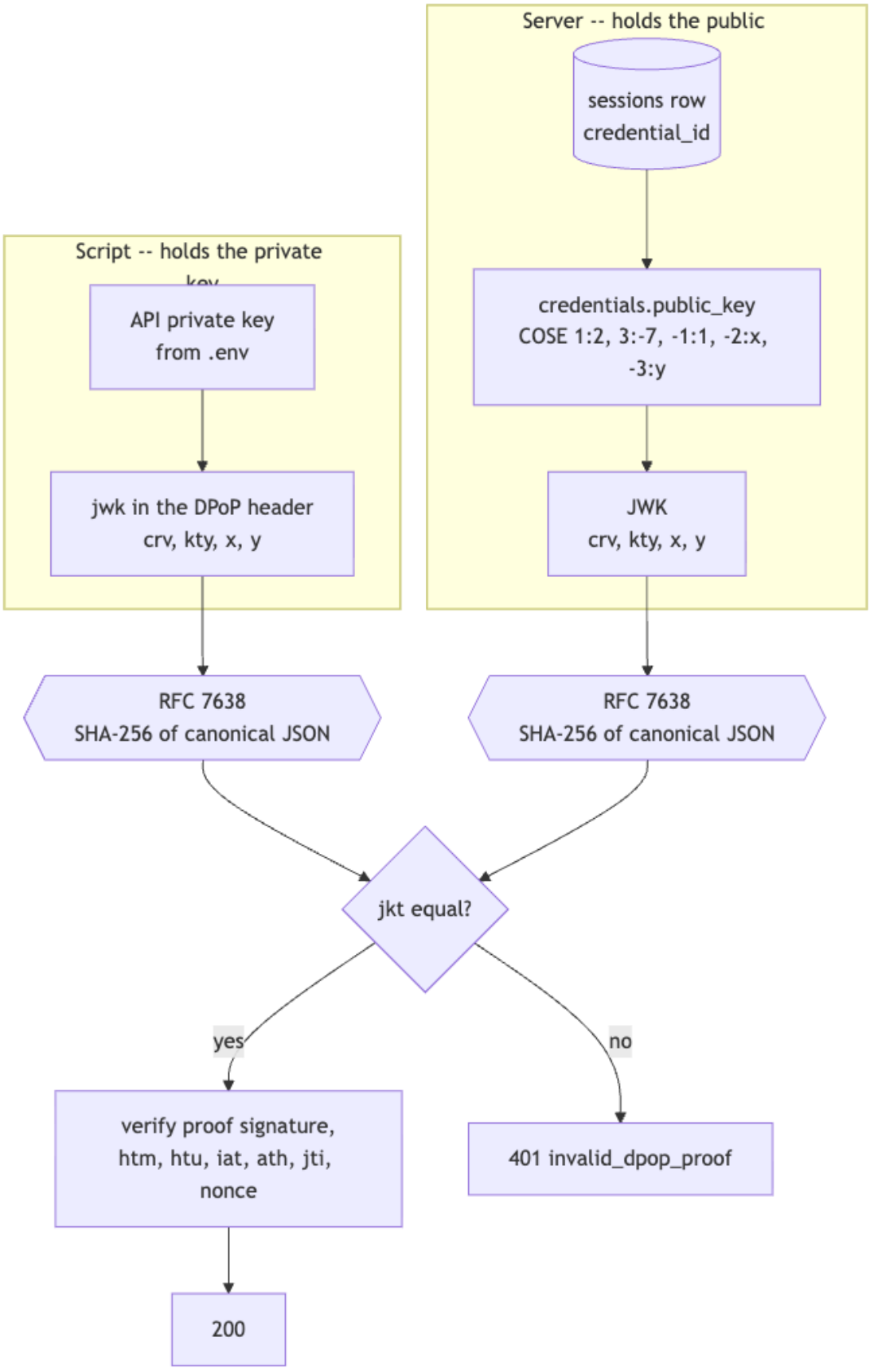


4.2 Assembling the assertion



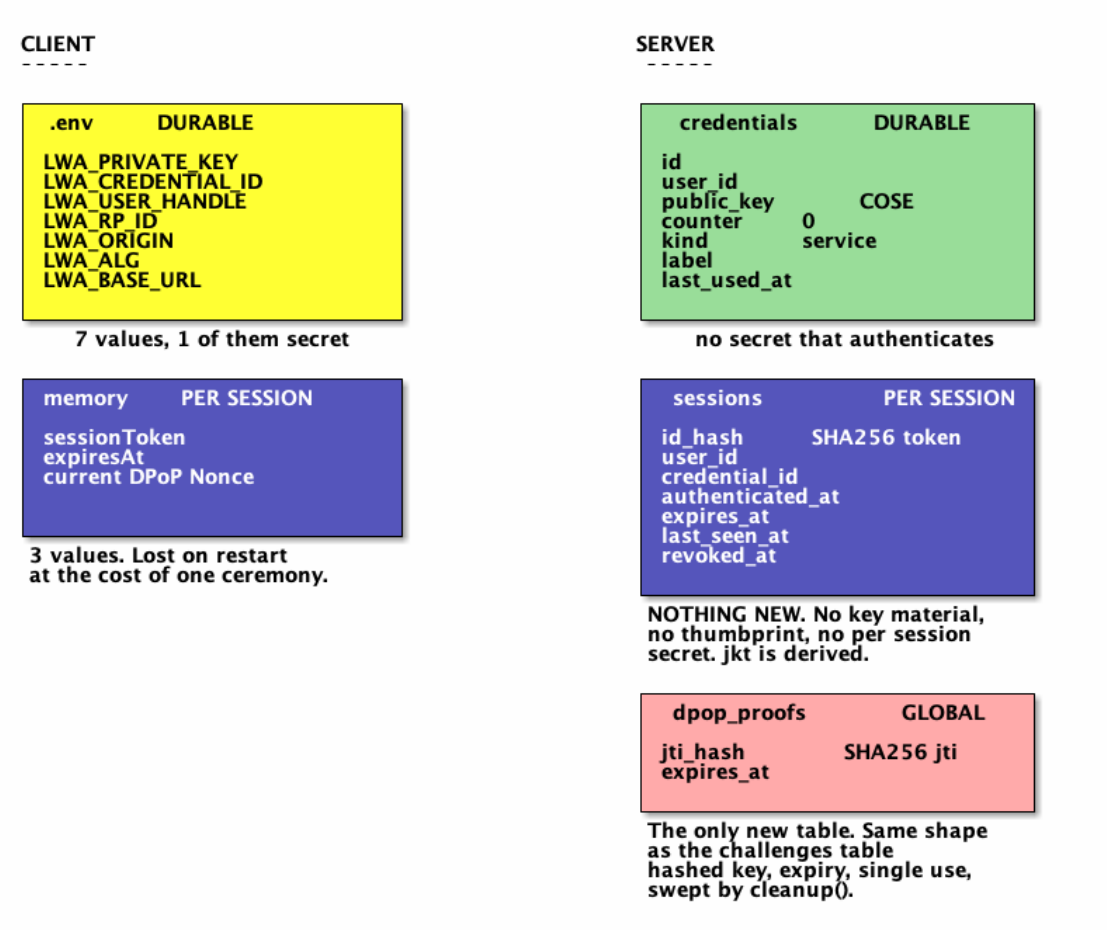
4.3 Why the server needs no new per-session state

The API Passkey key and the DPoP key are the same key. The server already holds that key's public half in `credentials.public_key`, and the session row already records `credential_id`. So the expected thumbprint is *derived*, never stored:


Server -- holds the public
sessions row
credential_id
credentials.public_key
COSE 1:2, 3:-7, -1:1, -2:x,
-3:y
JWK
crv, kty, x, y
Script -- holds the private
API private key
from .env
jwk in the DPoP header
crv, kty, x, y
RFC 7638
SHA-256 of canonical JSON
RFC 7638
SHA-256 of canonical JSON
jkt equal?
yes
no
verify proof signature,
htm, htu, iat, ath, jti,
nonce
401 invalid_dpop_proof
200

The check reduces to: *this proof was signed by the same key that produced the assertion that opened this session.* No `dpop_jkt` column, no issuance step, nothing generated per session. This is the direct answer to whether anything beyond the standard Passkey server-side confirmation is required: it is not.

4.4 What each side remembers



4.5 The DPoP proof

```
header  = {"typ":"dpop+jwt","alg":"ES256",
           "jwk":{"crv":"P-256","kty":"EC","x":"...","y":"..."}}
payload = {"jti":"<16 random bytes, base64url>",
           "htm":"POST",
           "htu":"https://app.example.com/api/reports",   -- no query, no fragment
           "iat":<unix seconds>,
           "ath":"<base64url(SHA-256(sessionToken))>",
           "nonce":"<last DPoP-Nonce seen>"}

signing input = base64url(header) || "." || base64url(payload)
signature     = ES256 over that -- RAW r||s, NOT DER

POST /api/reports HTTP/1.1
```

```
Host: app.example.com
Authorization: DPoP <sessionToken>
DPoP: eyJ0eXAiOiJkcG9wK2p3dCIs...
```

Server-issued nonces (RFC 9449 section 8) are worth enabling. They are the one control aimed squarely at a hostile-but-legitimate key holder: the client cannot pre-generate a proof for later use, because it cannot sign a nonce the server has not yet issued.

4.6 Concurrency and session lifetime

Unlimited simultaneous sessions, today, with no changes: `completeAuthentication` inserts a fresh row per ceremony and sessions share no mutable state.

Why `signCount` must be zero. With the counter pinned at `0` the compare-and-swap is `0 -> 0` and parallel ceremonies never contend. A strictly increasing counter would make concurrent sessions fight over one CAS, and the losers would get `409`. The requirement for many simultaneous sessions therefore *forces* `signCount = 0`. It also matches the human's Apple Passkey, which reports `0` forever, so any such deployment already tolerates it.

On binding a session to a TCP connection. Tempting, and not implementable. The client cannot see connection boundaries – Node's `fetch` pools and reuses connections through undici, and HTTP/2 multiplexes. The server cannot see them either: behind any reverse proxy it observes the *proxy's* connection. The one standard that truly binds to the connection, RFC 9729 Concealed HTTP Authentication, needs TLS-exporter access unavailable behind a terminating proxy, and emits an *identical* `Authorization` header for every request on that connection, so it is replayable within it.

What connection-scoping reaches for is a small blast radius for leaked session material. A per-request DPoP proof delivers that more completely: a captured token is useless without the key on **every** request, whatever the connection does. Pair it with a short `sessionAbsoluteMs` for the `service` kind and re-ceremony on `401`.

5 Implementation Notes

5.1 The signature-format trap

The same key signs in two incompatible encodings, and getting it wrong produces a verification failure with no diagnostic:

Context	Standard	ECDSA signature encoding
WebAuthn assertion	COSE alg `-7`	ASN.1 DER `SEQUENCE { r, s }`
DPoP proof (JWS)	RFC 7518 `ES256`	raw =r s=, 64 bytes

WebCrypto's `ECDSA` sign returns the raw form, so the WebAuthn path must convert to DER and the DPoP path must not. Ed25519 (`-8` / `EdDSA`) sidesteps this entirely – raw 64 bytes in both – and is worth preferring wherever the key store supports it.

5.2 Host routing

Machine routes must mount **outside** the starter's origin-check middleware. A script sends no `Origin` header, so `isExactOrigin(null, ...)` rejects it. That check exists to stop cross-site request forgery, which is a browser-only threat: CSRF needs ambient credentials a browser attaches automatically, and a script has none. Dropping it for machine routes loses nothing; the challenge and the DPoP proof carry the security there.

Cookies are likewise unused on machine routes – `challengeToken` and `sessionToken` travel in the JSON body and the `Authorization` header. No service change is needed for this: `authenticationOptions()` already *returns* `challengeToken` as a value, and only the browser routes choose to wedge it into a cookie.

5.3 What this design does not need

- No OAuth authorization server, no authorization codes, no token endpoint.
- No JWT *issuance*. The only JWT is the client-made DPoP proof, verified with the key it embeds – so no server signing key, no JWKS endpoint, no key rotation.
- No `dpop_jkt` column; derived from `credentials.public_key`.
- No per-session server secret.
- No attestation verification: `fmt: "none"`, and nothing a signer asserts about itself is evidence anyway.

6 Open Items

1. **Machine sessions must not enroll credentials.** `registrationOptions({ sessionToken })` authorizes registration from any live session. The script's session is a live session, so a leaked `.env` key could register a spare credential and survive revocation of the first – revocation stops being a remedy. Default to refusing session-path registration when the authorizing session's credential `kind` is non-null; rotation then goes back through a human-authorized grant.

2. **Kind-scoped last-credential guard.** Without it the script's credential counts as "another active credential", letting you revoke the human's only Passkey while reporting success.

3. **Versioned migrations.** `LOCALWEBAUTHN_SCHEMA_VERSION` is `1` and the migration runner is `CREATE TABLE IF NOT EXISTS`, which cannot add a column. Prerequisite for every schema change here.

4. **Per-kind session durations**, so `service` sessions can be minutes while human sessions stay hours.

5. **Kind-scoped pending-grant index** – only if the bootstrap-token variant is also supported. The flow above uses the session path and issues no grant.

7 Cross-References

- `docs/API-AUTH/DESIGN-MACHINE-CREDENTIALS.md` – the `kind` column, the byte formats, threat model against API keys, fleet topology.
- `docs/API-AUTH/CONSENSUS.md` – reconciles three designs; surveys RFC 9449 (DPoP), RFC 9421 (HTTP Message Signatures), RFC 9729 (Concealed), and `draft-ietf-oauth-first-party-apps`; argues for the mechanism over the OAuth machinery.
- `docs/API-AUTH/PROVISIONING.md` – the bootstrap-token variant, where the script rather than the browser generates the key pair.
- Issue #11 – the `kind` column this design depends on.